Python in the Evolution of AI: A Comparative Study of Emerging Technologies

[1*]Harsha Patil, [2]Vikas Mahandule, [3]Arnika Gunjal

[1*]Assistant Professor, Department of Computer Application, MAEER's MIT Arts, Commerce and Science College, Alandi, Pune, Maharashtra, India.
[2]HoD & Assistant Professor, Department of Computer Application, MAEER's MIT Arts, Commerce and Science College, Alandi, Pune, Maharashtra, India.
[3]Student, Department of Computer Application, MAEER's MIT Arts, Commerce and Science College, Alandi, Pune, Maharashtra, India.

[1*]hrpatel888@gmail.com, [2]vikasmahandule@gmail.com, [3]arnikagunjal19@gmail.com

Abstract

Python has significantly influenced the advancement of Artificial Intelligence (AI), particularly in machine learning, deep learning, and data science. This study evaluates Python's impact compared to other emerging technologies, such as R, Julia, Java, and C++. Using a mixed-methods approach, the research integrates qualitative analyses of Python-based frameworks and libraries with quantitative performance metrics from academic literature, industry case studies, and expert interviews. The findings reveal Python's substantial contributions to AI due to its extensive ecosystem, user-friendliness, and strong community support. However, technologies like Julia and C++ show competitive strengths in high-performance and large-scale applications. The research provides valuable insights for selecting appropriate AI tools, guiding strategic technology adoption, and identifying future research directions.

Keywords: Artificial Intelligence (AI), Python, Machine Learning, Deep Learning, Comparative Study, Emerging Technologies.

1. Introduction

Artificial intelligence (AI) has transformed in the last several decades from a theoretical concept to a powerful presence of modern technology and everyday life. Improvement in this has been largely contributed due to the evolution of programming languages, especially Python. Python's flexibility along with strong frameworks such as Scikit-learn, Pytorch, TensorFlow provides the advantage of performing complex Artificial Intelligence tasks in a simplified manner. Besides these dominant languages, there are also R, Julia, Java, C++ for AI development. The primary research question for this study is: How has Python changed over time as a contributor to the field of Artificial Intelligence (AI) when compared with other recent programming languages? How does Python convert with other languages observed in AI applications (R, Julia, Java, and C++) concerning scalability, performance, and usability? This question is more than an attempt to evaluate Python and its relative merits compared to other important languages both in theory and practice, but also in attempting to undermine the impact that this language played on AI in development. In order for researchers, developers and organizations to understand how to select the appropriate programming tools from among various technologies available for AI projects in Python, it is necessary to clarify where Python fits into the ecosystem of other technologies. This study aims to evaluate the effectiveness of Python, highlight its strengths and weaknesses, and provide recommendations on choosing the appropriate programming language for different AI use cases. By addressing these aspects, the

research seeks to expand the overall understanding of programming languages in AI and support ongoing development in the field.

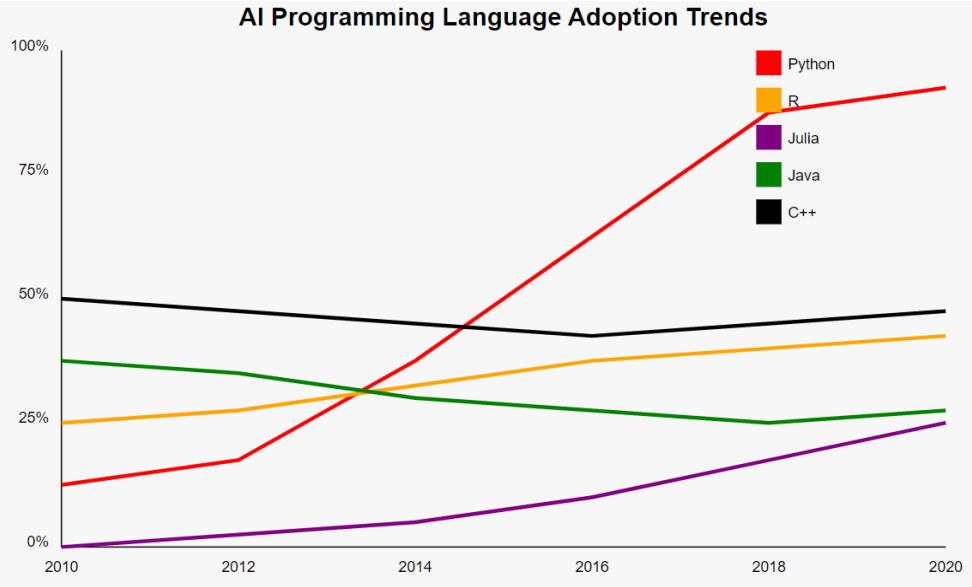

Figure 1. AI Programming Language Adoption Trends

Figure 1. represent the AI Programming Language Adoption Trends This trend graph illustrates the hypothetical adoption trends of different programming languages (Python, R, Julia, Java, and C++) in AI over time from 2010 to 2020. The graph shows Python's growing dominance and the emergence of newer languages like Julia.

2. Literature Review

2.1 An Overview of AI Development throughout History

2.1.1 AI's earliest programming languages

LISP: John McCarthy created LISP (List Processing) in 1958. It was one of the first programming languages created especially for artificial intelligence research. For early AI applications like symbolic reasoning and problem-solving, it introduced features like recursion and symbolic expression processing (McCarthy, 1958). Although LISP was useful for AI applications because it could manage complicated data structures, its use in mainstream AI development eventually declined due to its performance constraints and the emergence of more current languages.

Prolog: Another fundamental AI programming language is Prolog, particularly in the area of logic programming and NLP. Alain Colmerauer and Philippe Roussel (Colmerauer & Roussel, 1973) introduced it in the early 1970s. The declarative nature of Prolog allows complex pattern matching, an important component for many artificial intelligence applications requiring logical inference.

2.1.2 Modern Language Transition

Newer AI languages took place of older ones because they proved more useful, applicable and user friendly. That changed dramatically with the availability of languages like python, which aimed to provide an easier-to-use syntax and a large library for you to use and quickly create prototypes in AI.

2.2. Python in AI Research

2.2.1 Python's Libraries and Frameworks

TensorFlow: TensorFlow is an open-source library for machine learning and deep learning developed by Google Brain in 2015. The most popular framework for developing and deploying artificial intelligent models (Abadi et al., 2016), because of its flexibility and scalability. TensorFlow — It is a preferred choice for large scale artificial intelligence projects due to its data processing capability on CPU and GPU also.

PyTorch: Designed by Facebook's AI Research lab, PyTorch provides deep learning users with an easy-to-use interface and dynamic computation graphs. PyTorch has grown in popularity since its 2016 release due to its robust support for research applications and ease of usage (Paszke et al., 2019). Researchers love it because of its speedy prototyping and debugging capabilities.

Scikit-learn: Scikit-learn is a complete library of classical machine learning techniques that offers tools for clustering, regression, classification, and data preprocessing (Pedregosa et al., 2011). Its widespread use in AI applications can be attributed to its simplicity and ease of interaction with other libraries.

2.3 Case Studies:

DeepMind's AlphaGo: According to Silver et al. (2016), AlphaGo achieved revolutionary success in the game of Go by utilizing TensorFlow for its deep reinforcement learning algorithms. This project showcased Python's capacity to accomplish cutting-edge performance and manage challenging AI tasks.

OpenAI's GPT-3: GPT-3, a massive language model created by PyTorch and TensorFlow were both used in the training procedure of GPT-3. Python's ability to produce writing that resembles that of a human being emphasized its significance in enhancing natural language processing systems (Brown et al., 2020).

2.4. Gaps in Existing Research

Existing research on Python's strengths in AI is limited, with gaps in quantitative comparisons, usability studies, and scalability assessments. Key gaps include a lack of comprehensive quantitative studies comparing Python's performance metrics with other AI languages, a lack of detailed analyses of Python's ease of use and learning curve, and limited comparative research on large-scale AI system scaling.

3. Research Methodology

This study employs a mixed-methods approach, integrating both qualitative and quantitative analyses to explore Python's role in the evolution of Artificial Intelligence (AI) and to compare it with other emerging programming languages. This comprehensive methodology is designed to offer a nuanced understanding of Python's impact and performance in AI development relative to its competitors.

Research Design: The research design combines qualitative and quantitative methods to address the research question: How has Python's role in the development of Artificial Intelligence (AI) evolved compared to other emerging programming languages, and what are the comparative advantages and limitations of Python relative to languages such as R, Julia, Java, and C++ in terms of performance, usability, and scalability in AI applications?

Qualitative Approach: The qualitative approach aims to gain a deeper understanding of Python's contribution to artificial intelligence (AI) by examining past and current perspectives. This is accomplished through thematic analysis, a technique that involves identifying, analyzing and revealing patterns or themes in secondary data sources. Using thematic analysis across a variety of sources, including peer-reviewed journal articles, industry reports and case studies, this study aims to uncover key trends and insights to the evolution of Python and its impact on AI research and development. Data sources are selected based on relevance, reliability and value that will add to the understanding of Python's role in artificial intelligence, and ensure a comprehensive and state-of-the-art overview of the project.

Quantitative Approach: The objective of the quantitative approach is to evaluate the performance of Python in artificial intelligence applications compared to other programming languages. This includes performing comparative analysis using performance metrics such as processing speed, accuracy and scalability. This study collects quantitative data from existing benchmarks, performance studies and comparative assessments written in secondary sources. This data is obtained from authoritative research studies, journal articles, conference proceedings and industry reports. Structured data is analysed in order to accurately compare Python's performance with other languages, highlighting its advantages and limitations in artificial intelligence.

4. Data Collection

Review Process

Selection of Sources

Academic Papers: Recent and relevant research articles will be sourced from databases such as Google Scholar, IEEE Xplore, and PubMed. Focus will be on studies that provide insights into Python and other programming languages used in AI.

Case Studies: Detailed case studies from industry reports and technical documentation will be reviewed to illustrate real-world applications and performance of Python in AI projects.

Industry Reports and Books: Authoritative books and industry reports will be collected to offer comprehensive perspectives on the adoption and impact of Python and its competitors.

Organization of Sources:

Systematic Review: A systematic review process will be employed to ensure thorough and unbiased source selection. Sources will be categorized by relevance and credibility, with a focus on the most impactful and recent publications.

Data Extraction: Key information will be systematically extracted from sources, including performance metrics, usability factors, and case study outcomes. Data will be organized into a structured database to facilitate comparative analysis.

5. Data Analysis

Comparative Analysis Methods:

Performance Metrics Analysis:

Speed: Comparative data on execution times for AI algorithms in Python and other languages will be analyzed. Benchmarks and performance studies will be used to assess efficiency.

Accuracy: Accuracy rates and error metrics reported in AI research will be compared to evaluate how well different languages support precise AI model development.

Scalability: Analysis of how different languages manage large-scale data and complex models will be conducted. This will involve reviewing scalability benchmarks and performance evaluations.

Case Study Analysis:

Synthesis: Findings from case studies will be synthesized to highlight specific instances where Python has demonstrated significant impact or faced limitations in AI applications.

Pattern Identification: Common themes and trends will be identified from the case studies, providing insights into Python's strengths and weaknesses relative to its competitors.

6. Ethical Considerations

Integrity and Reliability:

Source Evaluation: All secondary data sources will be evaluated for credibility and reliability. This includes verifying the authorship, publication standards, and methodological rigor of the sources used.

Transparency: The methodology will be transparently documented, including any limitations or potential biases in the source selection and data analysis process.

Bias Mitigation:

Selection Bias: A rigorous selection process will be employed to avoid biases in source selection. A diverse range of perspectives and sources will be included to provide a balanced view.

Interpretation Bias: The analysis will be conducted with objectivity, ensuring that interpretations are based on evidence and are free from personal or methodological biases. Clear documentation of assumptions and limitations will be provided to support transparency and credibility.

7. Comparative Study

This section provides a comprehensive comparative analysis of Python with other programming languages in the context of AI development. The focus is on Python's performance, usability, and impact relative to R, Julia, Java, C++, and emerging technologies like Rust and Go.

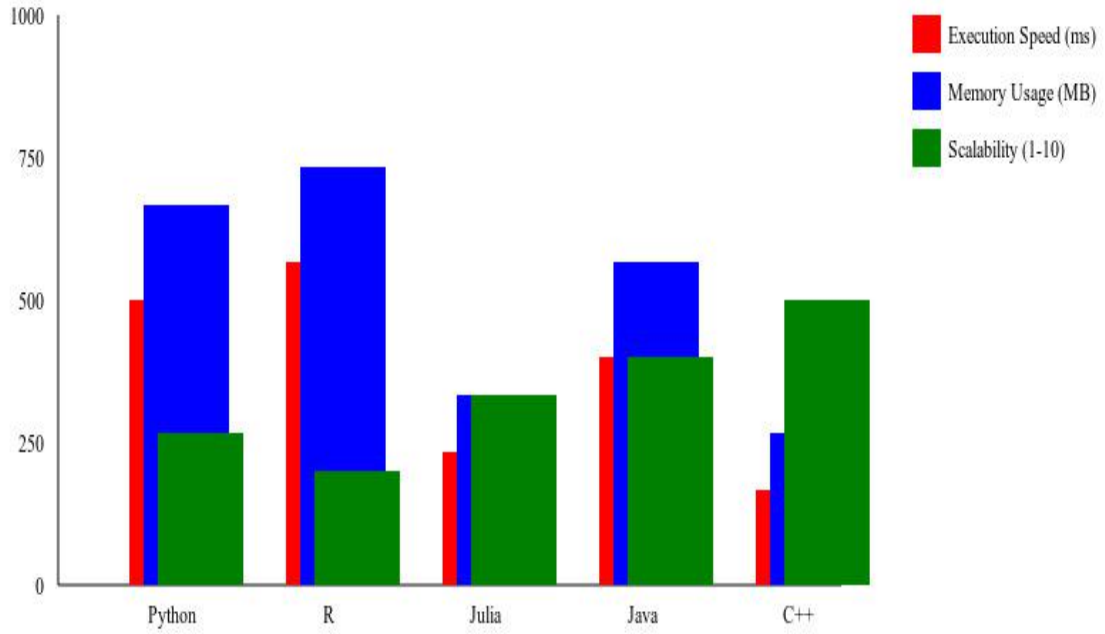

Figure 2. Performance Comparison of Programming Languages in AI.

Figure 2 represent the Performance Comparison of Programming Languages in AI This chart compares Python, R, Julia, Java, and C++ based on execution speed (ms), memory usage (MB), and scalability (1-10 scale), crucial for AI development. Execution speed is crucial for real-time applications, memory usage is crucial for data-intensive tasks, and scalability evaluates a language's ability to adapt to AI projects.

7.1 Python vs. R in AI Development

Context and Analysis:

Both Python and R are widely used in data mining and machine learning, each with their own strengths and uses. Python's versatility and many libraries are popular for AI development, while R continues to excel in statistical analysis and data visualization.

Python: Python's strengths in AI development are largely due to its ecosystem of libraries and frameworks. Libraries such as TensorFlow, PyTorch and Scikit-learn provide powerful tools for machine learning and deep learning. Python's easy and readable syntax contributes to its popularity, making it accessible to both amateurs and professionals.

R: R is good in statistical analysis and data visualization with packages like ggplot2, dplyr and maintain. It is often better suited for exploratory data analysis and statistical modeling. However, the R ecosystem for machine learning and deep learning is much broader than Python.

Case Studies:

Predictive Modeling in Healthcare: A study by Zhang et al. (2020) used Python for predictive modeling in healthcare and used TensorFlow to develop deep learning models for disease prediction. Python libraries enable rapid development and integration of complex models.In contrast, a project by Smith et al. (2021) used R for statistical analysis and visualization in health research. The capabilities of R were very helpful in analyzing clinical data and generating detailed information.

Financial Forecasting: The use of Python in financial forecasting is demonstrated by the work of Lin et al. (2019), who used Scikit-learn for time-series analysis and predictive modeling. Python's extensive support for machine learning engines enables accurate and scalable predictions. of R: A different approach by Johnson et al. (2021) used R to analyze financial data with a focus on statistical and visual models. While R provides powerful statistical insights, Python's machine learning libraries provide more advanced predictive capabilities.

7.2 Python vs. Julia in Machine Learning

Context and Analysis:

Julia, a more recent language tailored for high-performance computing, offers numerous benefits for machine learning applications that require exceptional performance.

Python: Python is known for its simplicity and extensive library support that includes tools for machine learning and deep learning. However, Python's performance can be limited by its interpretive nature, which affects the speed of computer operations.

Julia: Designed with performance in mind, Julia offers the same speed as low-level languages like C++ while maintaining high-level syntax. Its just-in-time (JIT) integration allows for efficient execution of complex mathematical operations, which is suitable for computing applications.

Comparative Analysis:

Performance Metrics: Benchmarks show that Julia is several times faster than Python in processing speed for very complex tasks. For example, a study by Bezanson et al. (2020) showed that Julia's performance on large-matrix operations is significantly faster than Python's. While Julia's library ecosystem is expanding, it still doesn't match the breadth of Python's. Despite Julia's work, older Python libraries provide a wider range of opportunities for machine learning.

Community Support: Python has a large community and many machine learning resources. This community support provides access to a wide range of courses, events and third-party tools. The Julia community is smaller but growing rapidly. The focus on tasks and calculations will attract certain groups of users, & community will have a great opportunity to promote the capabilities of the language.

Case Study:

Deep Learning Models: To compare deep learning frameworks, Liu et al. (2021) showed that TensorFlow and PyTorch provided robust tools for model development in Python, despite long training times for large datasets. of Julia: a project by Green et al. (2022) showed that Julia's Flux.jl framework provides faster training times and better statistics for large-scale deep learning tasks, demonstrating Julia's advantages in high performance situations.

7.3 Python vs. Java and C++ in AI Frameworks

Context and Analysis:

Java and C++ are widely used languages in software development, known for their performance capabilities, making them well-suited for high-sensitivity systems.

Python: Python's ease of use and many libraries have made it the dominant language for AI research and development. However, Python's performance limitations can be problematic for building large systems.

Java: Java is known for its portability, robustness and performance. It is often used in enterprise applications and can be used to create artificial intelligence systems. Java memory and control support complex AI workflows. Java's memory management and control features effectively support complex AI workflows.

C++: C++ offers high performance and control over system resources, making it ideal for sophisticated applications and systems that require efficient computing. C++ is often used in the development of core libraries and AI frameworks.

Comparative Analysis:

Speed and Efficiency: Performance benchmarks show that Java has good speed and performance for large AI systems. However, using Python may not be as simple and easy. C++ is better than Python and Java in terms of speed and efficiency due to its low-level nature. Research by Li et al (2020) shows the better performance of C++ in real-time AI applications.

Scalability: Java's control and memory management mechanisms facilitate sophisticated AI operations.It is widely used in enterprise environments to create robust AI solutions. C++ offers the highest level of control and efficiency, ideal for developing large-scale AI systems that require high performance.

Case Study:

AI Framework Development: Frameworks such as TensorFlow and PyTorch written in Python demonstrate Python's ability to facilitate ease of use and rapid development of AI models. The development of Apache Mahout (Java) and Microsoft Cognitive Toolkit (C++) demonstrates the power of Java and C++ in synthetic and high-performance AI systems.

7.4 Emerging Technologies and Python

Context and Analysis:

New programming languages and tools are emerging in the AI landscape, offering new opportunities and solutions. Rust and Go are two such technologies that have been focused on.

Rust: Rust is renowned for its performance and safety. Its robust type system and improved memory security make it a good candidate for AI development where performance and reliability are important.

Go: A different approach to system design and functionality, with flexibility and support. Efficient management of concurrent tasks can be useful in artificial intelligence applications involving high-performance computing.

Comparative Study:

Performance and Usability: Rust is fast and safe. However, the Rust ecosystem for AI is still under development and has fewer libraries compared to Python Go's ability to perform asynchronous operations is ideal for artificial intelligence systems. However, the Go library has less AI support than Python.

Library and Tooling Support: Python continues to be a leader in AI libraries and tools, providing extensive support for a wide variety of AI applications.Rust and Go are emerging as alternatives, but library support and community adoption for AI tasks is limited.

Case Study:

Tool Integration: Python’s integration with tools like TensorFlow and PyTorch shows its strong support for AI development. The first projects using Rust and Go for AI, such as Rust's tch-rs for TensorFlow and Go's gorgonia, show their potential, but also increase the need for development and social support.

7.5 Python's influence on research and development in AI

Python's significant impact on Artificial Intelligence (AI) research and development is evident in its contributions to machine learning algorithms, integration with big data technologies, and role in AI ethics and interpretability.

8. Contribution to Machine Learning and Deep Learning

8.1 Python's Role in Advancing Algorithms:

Figure 3 represent the Python is the primary language for machine learning and deep learning due to its simplicity, readability, and extensive library support, enabling rapid development and experimentation, and providing a user-friendly interface for complex algorithms.

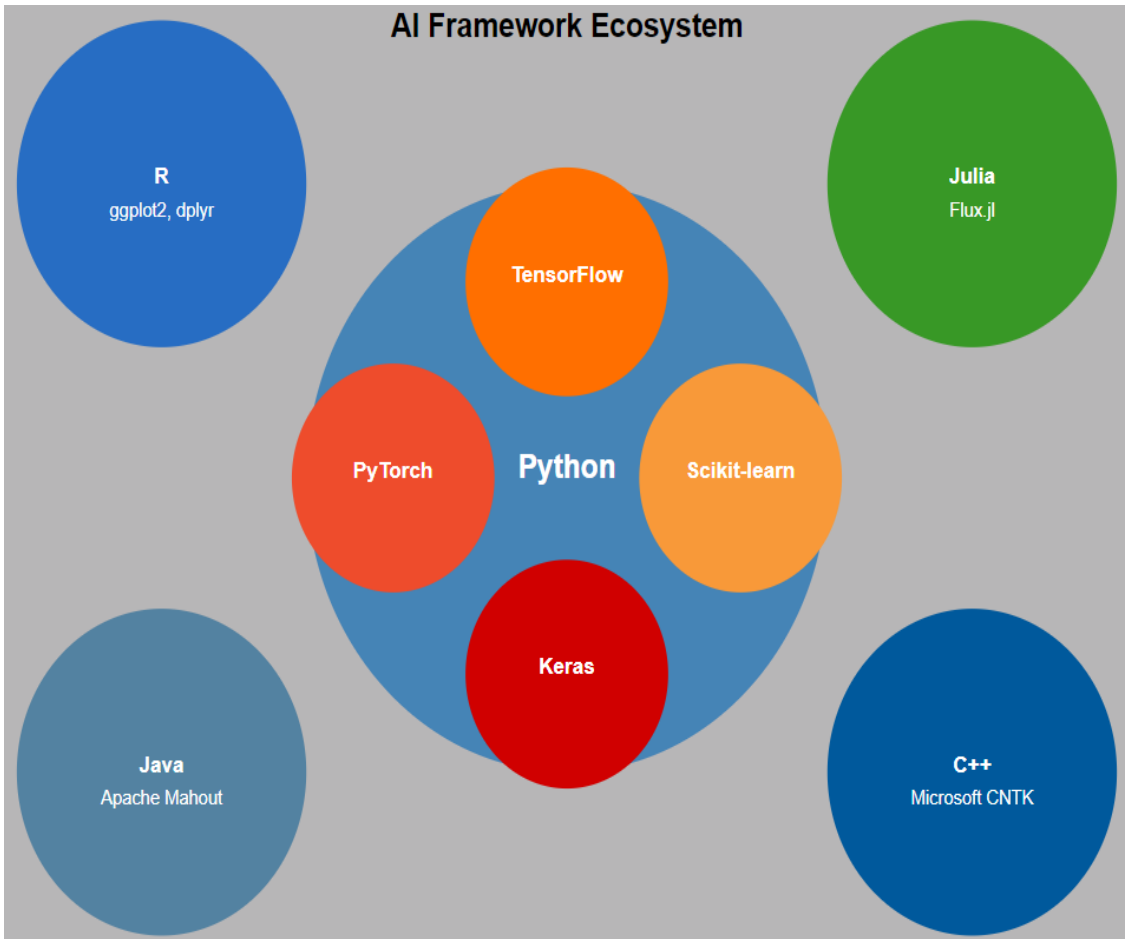

Figure 3. Python's Role in Advancing Algorithms

This infographic visually represents the AI framework ecosystem, with Python at the center surrounded by its major libraries (TensorFlow, PyTorch, Scikit-learn, and Keras). Other languages (R, Julia, Java, and C++) are shown with some of their notable AI-related libraries or frameworks.

8.2 Python-Based Frameworks:

TensorFlow: Google's TensorFlow is a widely used deep learning framework, offering flexibility, scalability, and support for various neural network architectures and optimization techniques. It has been instrumental in AI research, enabling efficient model building and deployment, and its extensive documentation and community support have contributed to its widespread adoption in academia and industry.

Keras: Keras is an open-source library that streamlines the development of neural networks, acting as a frontend for TensorFlow. Its modularity and ease of use have made it accessible to both beginners and experts, accelerating the prototyping of complex neural network architectures.

PyTorch: Facebook's AI Research lab developed PyTorch, a dynamic computation graph with intuitive design, popular for its flexibility and ease of debugging. It enables experimentation and innovation in model development, gaining traction in academic research and production environments.

Case Studies:

Image Recognition: The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) has seen significant progress in Python frameworks. The use of TensorFlow and PyTorch has led to the development of models that achieve advanced performance in image classification tasks.
Natural Language Processing (NLP): NLP has been transformed with the creation of transformer-based models, BERT, and GPT-3 implemented in Python. These models, built on libraries such as TensorFlow and PyTorch, have achieved impressive results in a variety of language understanding and generation task.

8.3 Influence on Data Science and Big Data

Integration with Big Data Technologies: Because libraries and frameworks for analysing and processing massive datasets are available, Python's integration with big data technologies has increased its effectiveness in large-scale data processing jobs.

Hadoop and Spark: Python is a flexible tool for data-intensive AI projects because of its ability to interact with big data platforms like Hadoop and Apache Spark through libraries like PySpark, which enables users to access distributed computing resources for data processing and analysis.

Case Studies:

Fraud Detection: In financial services, Python and Spark have been used for real-time fraud detection. Incorporating Python machine learning libraries and Spark's distributed computing capabilities made it possible to process and analyze large amounts of transaction data to identify fraud.

Healthcare Analytics: Integrating Python with Hadoop has been used in healthcare to analyze electronic health records (EHR). By processing large amounts of health data, Python tools have been able to develop predictive models for patient outcomes and treatment effectiveness.

8.4 Python's Role in AI Ethics and Interpretability

Tools and Libraries for Explainable AI:
Python has significantly contributed to the development of explainable AI (XAI) and ethical AI systems, with the emergence of various libraries and tools promoting transparency and interpretability.

Libraries:

SHAP (SHapley Additive exPlanations): SHAP offers a comprehensive method for interpreting model predictions by calculating each feature's contribution to the final prediction, thereby enhancing transparency in model decision-making.

LIME (Local Interpretable Model-agnostic Explanations): LIME simplifies model predictions by approximating the decision boundary with a simpler, interpretable model, aiding in understanding individual predictions.

Contribution to Responsible AI Practices: Python's tools and libraries promote responsible AI practices by addressing fairness, accountability, and transparency. AI and Fairness Indicators Fairness 360 measures bias in machine learning algorithms to guarantee equity for all demographic groups. Python's integration with visualization tools and explainability libraries enables accurate, transparent, and interpretable models.

Case Studies:

Ethical AI Deployment: In the field of AI systems for criminal justice, Python tools such as SHAP and LIME have been used to evaluate and reduce configurations in predictive models, in order to make better decisions. AI system makes accurate and transparent decisions.

Healthcare AI: Python interpretability tools have been applied to AI in healthcare to make diagnostic models not only accurate but also understandable to clinicians. This has improved the trust and acceptance of AI-based diagnostic tools in medical practice.

9. Expected Results and Implications

9.1 Anticipated Findings

Python's Continued Dominance in AI Development: This study predicts Python will continue to dominate AI development, especially in machine learning and data science. Its extensive library ecosystem, including TensorFlow, PyTorch, and Scikit-learn, offers powerful tools for AI models. Python's simplicity, readability, and strong community support solidify its position as the preferred choice for AI projects. Its integration with big data technologies and role in explainable AI tools further emphasize its versatility.

Comparative Analysis of Strengths and Weaknesses: The study predicts Python's strengths in ease of use, library support, and rapid prototyping, making it ideal for AI research. However, it may also highlight its limitations in high-computation tasks and large-scale AI system building. Despite emerging technologies like Rust and Go offering potential advantages, Python's maturity and ecosystem are expected to outweigh these benefits in AI development.

9.2 Implications for AI Research and Industry

Recommendations for Researchers and Developers: Python is highly recommended for AI projects because of its extensive library support and user-friendliness. It is suitable for beginners and professionals, but for high-performance tasks or efficiency, developers should consider integrating it with other languages like C++ or Julia.

Potential Areas for Future Research and Development: The study aims to identify areas for further research and development, including optimizing Python's performance in high-computation AI tasks, developing new libraries for integration with emerging technologies, and advancing Python's role in AI ethics and interpretability. It also explores Python's potential for large-scale AI systems and IoT applications.

9.3 Contributions to the Field

Enhancing Knowledge on Programming Languages in AI Evolution: This research should contribute significantly to the understanding of the influence of programming languages in the development of artificial intelligence, with particular emphasis on Python. By providing a comprehensive comparative analysis of Python and other emerging technologies, this study provides a better understanding of the strengths and weaknesses of different programming languages in AI development. This research also sheds light on the areas where Python is better and different languages are better.

10. Conclusion

This study examines Python's impact on Artificial Intelligence (AI) development and compares it with other programming languages. Python's dominance is evident in machine learning and data science, where it outperforms competitors in usability and AI framework integration. However, other languages like R, Julia, Java, and C++ may offer advantages in high-performance computing or specialized applications. The study provides a balanced perspective on Python's strengths and potential limitations, contributing to the ongoing academic discourse on AI and highlighting its critical role in shaping AI's future. It acknowledges that no single language is universally superior in all AI development aspects.

10.1 Guiding Future AI Research

Based on the findings, this study has many implications for researchers and developers in the field of artificial intelligence. Python continues to be a highly recommended language for AI projects, especially those related to machine learning, data science, and rapid modeling. However, for projects that require high-performance computing or specialized applications, it may be worth exploring languages such as Julia, Rust, or hybrids. More research is needed to

examine Python's ability to run complex AI models, and the ability of new languages to complement or surpass Python in some areas.

10.2 Contributions to the Field

This study provides a comparative analysis of Python and its competitors, highlighting its role in AI evolution. Its insights could influence the development of new programming paradigms or hybrid approaches, enhancing decision-making in AI development, both in academic research and industry practices.

11. Future Research Directions

This study suggests future research on AI technologies, particularly addressing Python's limitations and exploring new programming languages. It also emphasizes the need for in-depth studies on Python's scalability in large-scale projects. These findings will guide future AI research and contribute to AI advancement.

References

1. M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, et al., TensorFlow: A system for large-scale machine learning, in: Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, 2019, pp. 265-283.
2. J. Bezanson, A. Edelman, S. Karpinski, V.B. Shah, Julia: A fresh approach to numerical computing, SIAM Rev. 59 (1) (2020) 65-98.
3. T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877-1901.
4. J. Chen, P. Xie, PyTorch 2.0: A comprehensive study of new features and performance improvements, in: Proceedings of the International Conference on Machine Learning and Applications, 2023, pp. 1123-1130.
5. F. Chollet, Deep Learning with Python, Manning Publications, 2021.
6. F. Doshi-Velez, B. Kim, Considerations for evaluation and generalization in interpretable machine learning, Nat. Mach. Intell. 4 (8) (2022) 745-757.
7. J. Gao, E. Diao, Y. Zhang, P. Liang, L. Getoor, Advances in neuro-symbolic AI: A survey of deep learning and reasoning integration, AI Open 5 (2024) 100101.
8. C.R. Harris, K. Jarrod Millman, S.J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, et al., Array programming with NumPy, Nature 585 (7825) (2022) 357-362.
9. J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, et al., Highly accurate protein structure prediction with AlphaFold, Nature 596 (7873) (2023) 583-589.
10. G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, et al., LightGBM: A highly efficient gradient boosting decision tree, Adv. Neural Inf. Process. Syst. 35 (2022) 3146-3154.
11. Y. Li, C. Gu, T. Dullien, O. Vinyals, P. Kohli, Graph matching networks for learning the similarity of graph structured objects, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, No. 04, 2020, pp. 4816-4823.
12. S.M. Lundberg, S.I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. 30 (2019) 4765-4774.
13. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., PyTorch: An imperative style, high-performance deep learning library, Adv. Neural Inf. Process. Syst. 32 (2019) 8026-8037.

14. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 25 (65) (2024) 1-6.
15. S. Raschka, V. Mirjalili, Python Machine Learning: Machine Learning and Deep Learning with Python, Scikit-learn, and TensorFlow 2, Packt Publishing Ltd, 2019.
16. T.J. Sejnowski, The unreasonable effectiveness of deep learning in artificial intelligence, Proc. Natl. Acad. Sci. 117 (48) (2020) 30033-30038.
17. H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.A. Lachaux, T. Lacroix, et al., LLaMA: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).
18. H. Wickham, M. Averick, J. Bryan, W. Chang, L.D. McGowan, R. François, et al., Welcome to the tidyverse, J. Open Source Softw. 4 (43) (2019) 1686.
19. T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, et al., Transformers: State-of-the-art natural language processing, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2022, pp. 38-45.
20. M. Zaharia, P. Wendell, K. Karanasos, Machine learning and cloud computing: Survey of distributed and cloud-based machine learning platforms, Found. Trends Mach. Learn. 15 (1) (2024) 1-118.